Strings

This is the first of a series of 4 sessions on sequences.
We will be examining the string datatype here.

Strings

This is an immutable sequence of characters that is delimited either with a pair of single `'`, double `"` or triple single `'''` or triple double `"""` quotes.

The string type is the easiest and probably the most intuitive datatype to work with.

There is possibily a conversion to and from string to almost any other datatype.

Contents:

Creating a string by assigning a string literal to a variable

```
# creating a string by assigning a value to a variable


# strings can be created using single quotes, double quotes, or triple quotes
# single and double quotes can be used interchangeably, but they must match
# the string must be enclosed in the same type of quotes at both ends
```

```
# single quotes can contain double quotes inside, and vice versa
# triple quotes can be used for multi-line strings
# triple quotes can contain both single and double quotes inside without
escaping
# triple quotes can also span multiple lines without needing to escape
newlines

a = 'hello world'               # single quotes

b = "hello world"               # double quotes

c = '''hello world'''           # triple single quotes

d = """hello world"""           # triple double quotes

e = 'hello "world"'             # single quotes with double quotes inside

f = "hello 'world'"             # double quotes with single quotes inside

g = '''this string
spans multiple
lines '''                       # triple single quotes with multiple lines

h = '😈Narendra is \nEvil!!!'    # embedded a unicode and a meta (newline)
character
print(h)
```
```
😈Narendra is
Evil!!!
```

Escape Sequence

Before printing all string are checked for escaoe sequences.
An escape sequence is a combination of a backslash \ followed by a character, used to represent something special in a string.

```
print('It\'s fine')             # single quotes with an apostrophe inside
print('He said \"Hello\"')      # single quotes with double quotes inside
print('C:\\progs\\COMP100\\')   # single quotes with a backslash inside
print('Hello\nWorld')           # single quotes with a newline character
print("Hello\tWorld")           # double quotes with a tab character
print('Hello\rWorld')           # single quotes with a carriage return
character
print('Hello\bWorld')           # single quotes with a backspace character
print('Hello\vWorld')           # single quotes with a vertical tab character
print('Hello\fWorld')           # single quotes with a form feed character
```
```
It's fine
He said "Hello"
C:\progs\COMP100\
Hello
World
Hello   World
World
```

```
HellWorld
HelloWorld
HelloWorld
# Unicode and Byte Escape Sequences
# Unicode characters can be represented using escape sequences like \uXXXX or
\UXXXXXXXX
# where XXXX or XXXXXXXX is the hexadecimal code for the character.
# For example, \u03A9 represents the Greek letter Omega (Ω).
# Byte escape sequences can be used to represent characters in a specific
encoding, such as UTF-8.
# For example, b'\xE2\x82\xA1' represents the character '₡' in UTF-8
encoding.
print('\101')                                  # octal escape sequence for 'A'
print('\x41')                                  # hexadecimal escape sequence for
'A'
print('\u03A9')                                # Unicode escape sequence for
Omega (Ω)
print('\U0001F600')                            # Unicode escape sequence for
grinning face emoji (😀)
print('\N{GREEK CAPITAL LETTER OMEGA}')        # Unicode character name for
Omega (Ω)
print('')
A
A
Ω
😀
Ω
```

Creating a string by the str() Constructor

```
# creating a string using the str() constructor
# the str() constructor can convert various data types to a string
# it can convert integers, floats, lists, dictionaries, tuples, and sets to
strings
# an empty string can be created by assigning an empty value to a variable


s = str()
print(type(s))                          # Output: <class 'str'>
print(s)                                # empty string


def create_and_print_str(val):
    print(str(val))

a = 'Hello World'
create_and_print_str(a)                 # string from string

a = 123_456_789
create_and_print_str(a)                 # string from integer

a = 3.14
```

```
create_and_print_str(a)             # string from float


a = ['Hello World', 'Python', 'Programming']
create_and_print_str(a)             # string from list

a = {1: 'Hello', 2: 'World'}
create_and_print_str(a)             # string from dictionary

a = {'Hello', 'World'}
create_and_print_str(a)             # string from tuple

a = ('Hello World',)
create_and_print_str(a)             # string from set
<class 'str'>

Hello World
123456789
3.14
['Hello World', 'Python', 'Programming']
{1: 'Hello', 2: 'World'}
{'World', 'Hello'}
('Hello World',)
```

String Methods

String methods that returns a new string

```
a = '  narendra pershad  '              #notice the two leading and two
trailing spaces
print(a)

print(f'       len: {len(a)}.')
print(f'capitalize: {a.capitalize()}.') #does not seems to work (why?)
print(f'     title: {a.title()}.')
print(f'  swapcase: {a.title().swapcase()}.')
print(f'     upper: {a.upper()}.')
print(f'     lower: {a.lower()}.')
print(f'    lstrip: {a.lstrip()}.')
print(f'    rstrip: {a.rstrip()}.')
print(f'     split: {a.split()}.')
print(f'replace e with a: {a.replace('e', 'a')}.')
tofind = 'd'
print(f'{tofind} occurs at position: {a.find(tofind)}.')
tofind = 'z'
print(f'{tofind} occurs at position: {a.find(tofind)}.')
  narendra pershad
       len: 20.
capitalize:   narendra pershad  .
     title:   Narendra Pershad  .
  swapcase:   nARENDRA pERSHAD  .
     upper:   NARENDRA PERSHAD  .
```

```
     lower:   narendra pershad  .
    lstrip: narendra pershad  .
    rstrip:   narendra pershad.
     split: ['narendra', 'pershad'].
replace e with a:   narandra parshad  .
d occurs at position: 7.
z occurs at position: -1.
```

String methods that returns a bool

```
val = ['2', 'A', 'a', ' ']
func = 'isdigit isspace islower isupper isalpha isalnum'.split()

for v in val:
    for f in func:
        print(f"'{v}' {f}: {eval(f"'{v}'.{f}()")}")
    print()
'2' isdigit: True
'2' isspace: False
'2' islower: False
'2' isupper: False
'2' isalpha: False
'2' isalnum: True

'A' isdigit: False
'A' isspace: False
'A' islower: False
'A' isupper: True
'A' isalpha: True
'A' isalnum: True

'a' isdigit: False
'a' isspace: False
'a' islower: True
'a' isupper: False
'a' isalpha: True
'a' isalnum: True

' ' isdigit: False
' ' isspace: True
' ' islower: False
' ' isupper: False
' ' isalpha: False
' ' isalnum: False
```

Non-string methods that returns a new string

```
a = 'chr(65)'               # converts the argument to the ASCII equivalent -
> 'A'
print(f'{a} -> {eval(a)}')
```

```
a = 'bin(65)'                # converts the argument to the binary number
representation -> 1000001
print(f'{a} -> {eval(a)}')

a = 'oct(65)'                # converts the argument to the octal equivalent -
> 101
print(f'{a} -> {eval(a)}')

a = 'hex(65)'                # converts the argument to the hex equivalent ->
41
print(f'{a} -> {eval(a)}')

a = 'ascii("café")'          # converts the argument to the ascii equivalent -
> 'caf\xe9'
print(f'{a} -> {eval(a)}')
chr(65) -> A
bin(65) -> 0b1000001
oct(65) -> 0o101
hex(65) -> 0x41
ascii("café") -> 'caf\xe9'
#demonstrating that str are immutable types
print('Address of the string object')
a = 'hello '
print(f'memory address: {id(a):,}')
a += 'world'
print(f'memory address: {id(a):,}')
a = ''
print(f'memory address: {id(a):,}')

print('\nAddress of the list object')
b = ['hello']
print(f'memory address: {id(b):,}')
b += 'world'
print(f'memory address: {id(b):,}')
b.clear()
print(f'memory address: {id(b):,}')
Address of the string object
memory address: 2,003,361,710,944
memory address: 2,003,361,673,712
memory address: 140,704,902,465,216

Address of the list object
memory address: 2,003,361,708,736
memory address: 2,003,361,708,736
memory address: 2,003,361,708,736
#see https://www.w3schools.com/python/python_string_formatting.asp
```

Formatted output

```
imperial_gallon_to_liter = 4.54609
print(f'Gallon   Liters')
gal = 0.5
print(f'{gal:>6} = {gal * imperial_gallon_to_liter:.3f}')
```

```
gal = 1
print(f'{gal:>6} = {gal * imperial_gallon_to_liter:.3f}')
gal = 10
print(f'{gal:>6} = {gal * imperial_gallon_to_liter:.3f}')
gal = 100
print(f'{gal:>6} = {gal * imperial_gallon_to_liter:.3f}')
Gallon   Liters
   0.5 = 2.273
     1 = 4.546
    10 = 45.461
   100 = 454.609
```

Formatting Types

:< Left aligns the result (within the available space)

:> Right aligns the result (within the available space)

:^ Center aligns the result (within the available space)

:= Places the sign to the left most position

:+ Use a plus sign to indicate if the result is positive or negative

:- Use a minus sign for negative values only

: Use a space to insert an extra space before positive numbers (and a minus sign before negative numbers)

:, Use a comma as a thousand separator

:_ Use a underscore as a thousand separator

:b Binary format

:c Converts the value into the corresponding Unicode character

:d Decimal format

:e Scientific format, with a lower case e

:E Scientific format, with an upper case E

:f Fix point number format

:F Fix point number format, in uppercase format (show inf and nan as INF and NAN)

:g General format

:G General format (using a upper case E for scientific notations)

:o Octal format

:x Hex format, lower case

:X Hex format, upper case

:n Number format

:% Percentage format

```
a = 3.141_592_653
print(f'{a}')
print(f'{a:_}')
print(f'{a:g}')
print(f'{a:e}')
print(f'{a:n}')
print(f'{a:g}')
a = 0.007_297_352_564_3
print(f'{a:f}')
print(f'{a:.3f}')
print(f'{a:%}')
a = 1_024
#following only works with integer
print(f'{a:d}')
print(f'{a:x}')
print(f'{a:o}')
print(f'{a:b}')
print(f'{a:_}')
print(f'{a:,}')
print(f'{a: }')
print(f'{a:=}')
print(f'{a:+}')
print(f'{a:-}')
```

```
3.141592653
3.141592653
3.14159
3.141593e+00
3.14159
3.14159
0.007297
0.007
0.729735%
1024
400
2000
10000000000
1_024
1,024
 1024
1024
+1024
1024
```

Raw Strings

If you don't want python to check for escape sequences i.e. treat the string as a raw string, you must prefix it with an `r`

```
print(r'It\'s fine')
print(r'He said \"Hello\"')
print(r'C:\\progs\\COMP100\\')
It\'s fine
He said \"Hello\"
C:\\progs\\COMP100\\
```

Binary Strings

Binary strings is represented by bytes or bytearray type.
This is needed when you are dealing wth binary file, networking, encryption or any raw binary format such as image files

Binary string vs Normal string

Normal string in Python is a sequence of Unicode characters.
Binary string is a sequence of raw 8-bit values

Creating a binary string

There are at least 4 ways of obtaining a binary string

Using a byte literal

```
# Binary strings are created by prefixing the string with a 'b' or 'B'
binary_string = b'hello world'  # binary string with single quotes
print(binary_string)
b'hello world'
```

Using byte() constructor

```
b2 = bytes([65, 66, 67])
print(b2)
b'ABC'
```

Using bytearray() (mutable version)

```
b3 = bytearray([65, 66, 67])
print(b3)
bytearray(b'ABC')
```

By using encoding

```
# encoding and decoding between strings and bytes
# Strings can be encoded to bytes using the encode() method, and bytes can be
decoded to strings using the decode() method.

# encoding
s = 'hello world'
b = s.encode('utf-8')       # encode string to bytes using UTF-8
print(b)                    # prints: b'hello world'
```

```python
# decoding
s2 = b.decode('utf-8')       # decode bytes back to string using UTF-8
print(s2)                    # prints: hello world
```

```
b'hello world'
hello world
```

This is the second of a series of 4 notebooks on sequences.
WE will be looking at tuples in this notebook

Tuple

A tuple in Python is:

- an ordered, immutable, heterogeneous collection of values (objects).
- best used when you want a fixed collection of items that should not change.

Examples of common use cases:

- Coordinates (x, y)
- Database records
- Function return values

Contents:

Creating a tuple by assignment

```
#creating tuple by assignment

# Empty tuple
a = ()
print(a)

# Tuple with numbers
b = (9, 8, 7)
print(b)
```

```python
# Tuple with strings
c = ('x', 'y', 'z')
print(c)

# Mixed data types
d = (0, 'one', [2], 3.0, True)
print(d)

# For a single-element tuple, you need a comma
e = (1,)
print(e)    # Output: (1,)
```

```python
# for single element tuple, you need to add a comma
f = (1,)  # this is a tuple with one element
print(f)                                # Output: (1,)
# creating tuple from other functions
g = tuple(range(5))  # creates a tuple from a range object
print(g)                                # Output: (0, 1, 2, 3, 4)
#also
f = (b, c, d)
print(f)                                # tuple of tuples
```

```
()
(9, 8, 7)
('x', 'y', 'z')
(0, 'one', [2], 3.0, True)
(1,)
(1,)
(0, 1, 2, 3, 4)
((9, 8, 7), ('x', 'y', 'z'), (0, 'one', [2], 3.0, True))
```

Creating a tuple by using the constructor

You can create a tuple from any iterable using `tuple()`.

```python
# tuples from constructor
# you may pass any sequence to the constructor
s = tuple()
print(type(s))                      # Output: <class 'tuple'>
print(s)                            # empty tuple

a = 'Hello World'
print(f'{tuple(a)}')                # tuple from string


a = ['Hello World', 'Python', 'Programming']
print(f'{tuple(a)}')                # tuple from list

a = 'Narendra Pershad'.split()      # this method returns a list
```

```
print(tuple(a))                         # can you predict how many elements will
be in the tuple?

a = {1: 'Hello', 2: 'World'}
print(f'{tuple(a)}')                    # tuple from dictionary - it only takes
the keys

a = {'Hello', 'World'}
print(f'{tuple(a)}')                    # tuple from set

a = ('Hello World',)
print(f'{tuple(a)}')                    # tuple from tuple

a = tuple(range(10))                    #from a range
print(a)

c = tuple(range(10))[2:5]                   #from slicing an existing tuple
print(c)
d = tuple(x for x in range(10))             #tuple from a iterable expression
print(d)

# advance???
a = tuple(zip(range(2), range(4, 6)))   #from a two or more iterables
print(a)

# a = 123_456_789
# print(f'{tuple(a)}')                  # tuple from integer, an integer is not
iterable

# a = 3.14
# print(f'{tuple(a)}')                  # tuple from float, a float is not
iterable
<class 'tuple'>
()
('H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd')
('Hello World', 'Python', 'Programming')
('Narendra', 'Pershad')
(1, 2)
('World', 'Hello')
('Hello World',)
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
(2, 3, 4)
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
((0, 4), (1, 5))
```

Creating tuples in other ways

```
#creating more tuples
u = (a, b)                      #creates a nested tuples from two existing
tuples, similar to append()
print(f'u: {u}')
v = a + b                       #concatenates the items into a single tuple,
similar to extend()
```

```
print(f'v: {v}')
w = a * 3                       #repeat the tuple 3 times
print(f'w: {w}')
w += a                          #extend w by a
print(f'w: {w}')
u: (((0, 4), (1, 5)), (9, 8, 7))
v: ((0, 4), (1, 5), 9, 8, 7)
w: ((0, 4), (1, 5), (0, 4), (1, 5), (0, 4), (1, 5))
w: ((0, 4), (1, 5), (0, 4), (1, 5), (0, 4), (1, 5), (0, 4), (1, 5))
```

Some Common pitfalls:

- Forgetting the comma in a single-element tuple.
- Expecting .append() or .insert() like lists.
- Forgetting tuples are immutable.

Summary

- Tuples are ordered, immutable collections.
- Can contain heterogeneous elements.
- Useful for fixed data, unpacking, dict keys.
- Support indexing, slicing, and basic functions (min, max, sum, len).

```
def remove_dunder(f):
    all = dir(f)
    result = []
    for x in all:
        if not x.startswith('__'):
            result.append(x)
    return ', '.join(result)

print(remove_dunder(tuple()), '\n')
print(remove_dunder(range(5)), '\n')
print(remove_dunder(list()), '\n')
print(remove_dunder(str()), '\n')
print(remove_dunder(set()), '\n')
print(remove_dunder(dict()), '\n')
count, index

count, index, start, step, stop

append, clear, copy, count, extend, index, insert, pop, remove, reverse, sort

capitalize, casefold, center, count, encode, endswith, expandtabs, find, form
at, format_map, index, isalnum, isalpha, isascii, isdecimal, isdigit, isident
ifier, islower, isnumeric, isprintable, isspace, istitle, isupper, join, ljus
t, lower, lstrip, maketrans, partition, removeprefix, removesuffix, replace,
```

rfind, rindex, rjust, rpartition, rsplit, rstrip, split, splitlines, startswi
th, strip, swapcase, title, translate, upper, zfill

add, clear, copy, difference, difference_update, discard, intersection, inter
section_update, isdisjoint, issubset, issuperset, pop, remove, symmetric_diff
erence, symmetric_difference_update, union, update

clear, copy, fromkeys, get, items, keys, pop, popitem, setdefault, update, va
lues

This is the third of a series of 4 notebooks on sequences.
We will be looking at lists in this notebook

List

A list in Python is a mutable heterogeneous collection of values (objects). It is similar to arrays in other languages, but far more flexible.

Contents:

Creating a list

Using the constructor to create a list

This is the simplest way to obtain a list.

```
#using the list constructor
#creating list
a = list()                                  # creates an empty list
print(a)
```

```
b = list('Narendra')                          #this works with any iterable
e.g. string, list, tuple, dict etc
print(b)

c = list((0, 1, 2, 3))                        #from a tuple
print(c)

d = list([1, 2.2, '3'])                       #from another list
print(d)

e = list({'x', 'y', 'z'})                     #from a set
print(e)

f = list({'one': 1, 'two': 2, 'three': 3})    #from a dict (only the keys of
the dict)
print(f)

g = list(range(10))                           #from a range
print(g)
[]
['N', 'a', 'r', 'e', 'n', 'd', 'r', 'a']
[0, 1, 2, 3]
[1, 2.2, '3']
['y', 'z', 'x']
['one', 'two', 'three']
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Creating a list by assignment

```
a = []
print(a)

c = [9, 8, 7]
print(c)

d = ['x', 'y', 'z']
print(d)

e = [0, 'one', [2], 3.0, True]      # nested list
print(e)
[]
[9, 8, 7]
['x', 'y', 'z']
[0, 'one', [2], 3.0, True]
```

Creating a list by a method call

split() method

The split() method of the the string class returns a list of the separated items. By default, the seperator is `' '`, but you may specify your own.

```
a = 'Narendra Pershad'.split()              #this method returns a list
```

```
print(a)
['Narendra', 'Pershad']
```

sorted() method

The built-in sorted() method returns a new sorted list of the items of the sequence.

All the first level items must be comparable (of the same type).

```
a = 'Mary had a little lamb. Its fleece was as white as snow'
print(sorted(a))                #str to list

a = 'Mary had a little lamb. Its fleece was as white as snow'.split()
print(sorted(a))                #list to list

a = tuple('Mary had a little lamb. Its fleece was as white as snow'.split())
print(sorted(a))                #tuple to list
[' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', '.', 'I', 'M', 'a', '
a', 'a', 'a', 'a', 'a', 'a', 'b', 'c', 'd', 'e', 'e', 'e', 'e', 'e', 'f', 'h'
, 'h', 'i', 'i', 'l', 'l', 'l', 'l', 'm', 'n', 'o', 'r', 's', 's', 's', 's',
's', 't', 't', 't', 't', 'w', 'w', 'w', 'y']
['Its', 'Mary', 'a', 'as', 'as', 'fleece', 'had', 'lamb.', 'little', 'snow',
'was', 'white']
['Its', 'Mary', 'a', 'as', 'as', 'fleece', 'had', 'lamb.', 'little', 'snow',
'was', 'white']
#more ways of creating a lists
u = [a, b]                  # make a list from two existing list, similar to
append()
print(f'u: {u}')

v = a + b                   # similar to extend()
print(f'v: {v}')

w = a * 3                   # repetition
print(f'w: {w}')

w += a                      # extend in place
print(f'w: {w}')
u: [('Mary', 'had', 'a', 'little', 'lamb.', 'Its', 'fleece', 'was', 'as', 'wh
ite', 'as', 'snow'), ['N', 'a', 'r', 'e', 'n', 'd', 'r', 'a']]
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[5], line 5
      2 u = [a, b]                  # make a list from two existing list, sim
ilar to append()
      3 print(f'u: {u}')
----> 5 v = a + b                   # similar to extend()
      6 print(f'v: {v}')
      8 w = a * 3                   # repetition

TypeError: can only concatenate tuple (not "list") to tuple
```

Processing a list

```
c = [f'{x*x}'.zfill(3) for x in range(6)]         #list comprehension

for x in c:
    print(x)
000
001
004
009
016
025
```

Mutating methods

These methods belong to the list class. They change the list itself and does not normally return a value.

The only exception is `pop()`, which returns the removed value as well as it mutates the list

```
# we will work with the following variables
data = ['0', 1, 2.0, {3, 'four'}, [5, 6.0],(7,), range(2)]
string_data = 'Hello world!'
list_data = ['0', '3', '2', '1', '2']
set_data = {'one', 'two', 'three'}  # not a sequence, but still works with
len()
tuple_data = ('0', ['1', '2', '3', '4'], '4')
range_data = range(5)
```

Append

The `append(self, object, /)` method modifies the original list by adding the argument as a single item at the end. The length of the original list increases by exactly 1, no matter what you append.

N.B. You can append any object (string, list, tuple, set, range, dict etc.) to a list. The entire object is added as a single item to the end of the list.

```
tmp = data.copy()
tmp.append(string_data)           # adds item (str) to the end of the list tmp
print(tmp)

tmp = data.copy()
tmp.append(list_data)           # adds all the items of list b to the end of
the list tmp as a single item
print(tmp)

tmp = data.copy()
tmp.append(set_data)          # adds all the items of set c to the end of the
list tmp as a single item
print(tmp)

tmp = data.copy()
tmp.append(tuple_data)           # adds all the items of tuple d to the end of
the list tmp as a single item
```

```
print(tmp)

tmp = data.copy()
tmp.append(range_data)          # adds all the items of range e to the end of
the list tmp as a single item
print(tmp)
['0', 1, 2.0, {'four', 3}, [5, 6.0], (7,), range(0, 2), 'Hello world!']
['0', 1, 2.0, {'four', 3}, [5, 6.0], (7,), range(0, 2), ['0', '3', '2', '1',
'2']]
['0', 1, 2.0, {'four', 3}, [5, 6.0], (7,), range(0, 2), {'two', 'one', 'three
'}]
['0', 1, 2.0, {'four', 3}, [5, 6.0], (7,), range(0, 2), ('0', ['1', '2', '3',
'4'], '4')]
['0', 1, 2.0, {'four', 3}, [5, 6.0], (7,), range(0, 2), range(0, 5)]
```

Clear

The `clear()` method removes all the items of the current list.
The list length will be zero.

```
print(f'original: {tmp}')

tmp.clear()
print(f' cleared: {tmp}')
original: ['0', 1, 2.0, {'four', 3}, [5, 6.0], (7,), range(0, 2), range(0, 5)
]
 cleared: []
```

Extend

The `extend(self, iterable, /)` method also modifies the original list. It unpacks another iterable and appends its items at the end. The length of the list increases by the number of elements in the iterable.

This is used when you want to merge another sequence into your list.

```
tmp = data.copy()
tmp.extend(string_data)          # adds each item of str a to the end of the
list tmp
print(tmp)

tmp = data.copy()
tmp.extend(list_data)          # adds each item of list b to the end of the
list tmp
print(tmp)

tmp = data.copy()
tmp.extend(set_data)          # adds each item of set c to the end of the
list tmp
print(tmp)

tmp = data.copy()
```

```
tmp.extend(tuple_data)          # adds each item of tuple d to the end of the
list tmp
print(tmp)

tmp = data.copy()
tmp.extend(range_data)          # adds each item of range e to the end of the
list tmp
print(tmp)
['0', 1, 2.0, {'four', 3}, [5, 6.0], (7,), range(0, 2), 'H', 'e', 'l', 'l', '
o', ' ', 'w', 'o', 'r', 'l', 'd', '!']
['0', 1, 2.0, {'four', 3}, [5, 6.0], (7,), range(0, 2), '0', '3', '2', '1', '
2']
['0', 1, 2.0, {'four', 3}, [5, 6.0], (7,), range(0, 2), 'two', 'one', 'three'
]
['0', 1, 2.0, {'four', 3}, [5, 6.0], (7,), range(0, 2), '0', ['1', '2', '3',
'4'], '4']
['0', 1, 2.0, {'four', 3}, [5, 6.0], (7,), range(0, 2), 0, 1, 2, 3, 4]
```

Insert

The `insert(self, index, object)` method adds the argument at the given index.

If the position is larger than the list, it is inserted at the end.
The list length increases by one.

```
tmp = data.copy()
tmp.insert(0, string_data)          # adds the item in the first position of
the list tmp
print(tmp)

tmp = data.copy()
tmp.insert(1, list_data)          # adds the item in the second position of
the list tmp
print(tmp)

tmp = data.copy()
tmp.insert(2, set_data)          # adds the item in the third postion of the
list tmp
print(tmp)

tmp = data.copy()
tmp.insert(0, tuple)          # adds the item in the first postiion of the
list tmp
print(tmp)

tmp = data.copy()
tmp.insert(1, range_data)          # adds the item in the second position of
the list tmp
print(tmp)
['Hello world!', '0', 1, 2.0, {'four', 3}, [5, 6.0], (7,), range(0, 2)]
['0', ['0', '3', '2', '1', '2'], 1, 2.0, {'four', 3}, [5, 6.0], (7,), range(0
, 2)]
```

```
['0', 1, {'two', 'one', 'three'}, 2.0, {'four', 3}, [5, 6.0], (7,), range(0,
2)]
[<class 'tuple'>, '0', 1, 2.0, {'four', 3}, [5, 6.0], (7,), range(0, 2)]
['0', range(0, 5), 1, 2.0, {'four', 3}, [5, 6.0], (7,), range(0, 2)]
```

Pop

The `pop(self, index=-1, /)` method removes the last item from the list. If the optional argument is given, then the item is removed from that position.

The list length will be one less.

```
c = 'Mary has a little lamb. Its fleece was as white as snow.'.split()
print(c)
r = c.pop()                                 # the last item is taken off
print(f'removed: "{r}" result: {c}')

r = c.pop(1)                                # the second item is taken off
print(f'removed: "{r}" result: {c}')
```

```
['Mary', 'has', 'a', 'little', 'lamb.', 'Its', 'fleece', 'was', 'as', 'white'
, 'as', 'snow.']
removed: "snow." result: ['Mary', 'has', 'a', 'little', 'lamb.', 'Its', 'flee
ce', 'was', 'as', 'white', 'as']
removed: "has" result: ['Mary', 'a', 'little', 'lamb.', 'Its', 'fleece', 'was
', 'as', 'white', 'as']
```

Remove

The `remove(self. value, /)` method removes the first occurence of a given value from the list.

A ValueError will be raised if the value in not found.

The list length will be one less.

```
fruits = 'apple banana cherry banana figs grape banana'.split()
print(f'  Original list -> {fruits}')
to_remove = 'banana'
print(f'Removing {to_remove} -> ', end='')
fruits.remove(to_remove)
print(fruits)

# to_remove = 'plum'
# fruits.remove(to_remove)        # this will raise a ValueError because the
item is not present in the list
```

```
  Original list -> ['apple', 'banana', 'cherry', 'banana', 'figs', 'grape', '
banana']
Removing banana -> ['apple', 'cherry', 'banana', 'figs', 'grape', 'banana']
```

Reverse

The `reverse(self, /)` method reverse the order the item of the list.
The list length remains constant.

```
a = list('Arben is evil!')
```

```
print(f'original: {a}')

a.reverse()
print(f'reversed: {a}')
original: ['A', 'r', 'b', 'e', 'n', ' ', 'i', 's', ' ', 'e', 'v', 'i', 'l', '
!']
reversed: ['!', 'l', 'i', 'v', 'e', ' ', 's', 'i', ' ', 'n', 'e', 'b', 'r', '
A']
```

Sort

The `sort(self, /, *, key=None, reverse=False)` method, orders the item of the current list. The list length remains constant.

```
a = list('Arben is evil!')
print(f'original: {a}')

a.sort()
print(f'  sorted: {a}')

a.sort(reverse=True)
print(f' reverse: {a}')
original: ['A', 'r', 'b', 'e', 'n', ' ', 'i', 's', ' ', 'e', 'v', 'i', 'l', '
!']
  sorted: [' ', ' ', '!', 'A', 'b', 'e', 'e', 'i', 'i', 'l', 'n', 'r', 's', '
v']
 reverse: ['v', 's', 'r', 'n', 'l', 'i', 'i', 'e', 'e', 'b', 'A', '!', ' ', '
']
a = 'Mehrdad is a genius'.split()
print(f'original: {a}')

a.sort()
print(f'  sorted: {a}')


a.sort(key=len)
print(f'     len: {a}')

a.sort(key=lambda word: sum(1 for letter in word.lower() if letter in
'aeiou'))
print(f'   vowel: {a}')
original: ['Mehrdad', 'is', 'a', 'genius']
  sorted: ['Mehrdad', 'a', 'genius', 'is']
     len: ['a', 'is', 'genius', 'Mehrdad']
   vowel: ['a', 'is', 'Mehrdad', 'genius']
```

Non-mutating methods

These methods do not change the original list.

The `len()`, `index()`and the `count()` methods as well as slicing techniques will be covered in the notebook on Sequences.

Copy

The `copy()` method returns a new list containing all the items in the current list.

```
a = [1, 2, 3]
b = a.copy()     # makes a shallow copy

b.append(4)

print(a)         # [1, 2, 3]
print(b)         # [1, 2, 3, 4]
print(a is b)    # False (different objects)
[1, 2, 3]
[1, 2, 3, 4]
False
```

Difference between copy and assignment

In an assignment, both identifier refers to the same object, so any change applied to one reference will affect the other one.

```
a = [1, 2]
b = a

print('original lists')
print(a, '<->' , b)

print('\nchanging the first list')
a.append(3)

print('first list changed')
print(a, '<->' , b)

print('\nchanging the second list')
a.remove(1)

print('second list changed')
print(a, '<->' , b)
original lists
[1, 2] <-> [1, 2]

changing the first list
first list changed
[1, 2, 3] <-> [1, 2, 3]

changing the second list
second list changed
[2, 3] <-> [2, 3]
```

Shallow copy vs. Deep copy

- copy() is shallow: it only duplicates the list itself, not the nested objects.

- If the list contains mutable elements (like other lists), both lists still share references to those inner objects.

In an assignment, both identifier refers to the same object, so any change applied to one reference will affect the other one.

```
a = [[1, 2], [3, 4]]
b = a.copy()
print(a)  # [3, 4]]
print(b)  # [3, 4]]

print('\nAfter append')
b[0].append(99)

print(a)  # [[1, 2, 99], [3, 4]]  ← inner list changed in both
print(b)  # [[1, 2, 99], [3, 4]]
[[1, 2], [3, 4]]
[[1, 2], [3, 4]]

After append
[[1, 2, 99], [3, 4]]
[[1, 2, 99], [3, 4]]
```

If you need a true independent clone, use the copy module:

```
import copy

a = [[1, 2], [3, 4]]
b = copy.deepcopy(a)

b[0].append(99)

print(a)  # [[1, 2], [3, 4]]
print(b)  # [[1, 2, 99], [3, 4]]
[[1, 2], [3, 4]]
[[1, 2, 99], [3, 4]]
```

Advanced ways of producing a list

```
a = list(zip(range(2), range(4, 6)))     #from multiple iterables
print(a)
b = list(range(10))[2:5]                 #from slicing an existing list
print(b)
c = [f'0{x}' for x in range(10)]         #list comprehension
print(c)
[(0, 4), (1, 5)]
[2, 3, 4]
['00', '01', '02', '03', '04', '05', '06', '07', '08', '09']
```

Summary

- A list is a mutable, ordered, heterogeneous collection of objects.
- It is similar to arrays in other languages, but more flexible.

- They support a wide range of operations: creation, indexing, slicing, comprehensions, and many mutating methods.
- Key differences:
    - append vs extend
    - shallow copy vs deep copy
    - sort (in place) vs sorted (new list)

This is the fourth of a series of 4 notebooks on sequences.
Here we will be looking at the common operations that can be done on sequences.

Sequence

Strings, tuples, list and range are the most common seqences that you will work with in Python.

This notebook attempts to explore some operations, that is commonly applied to all (or atleast most) sequences.

Contents
Methods that are not attached to **ANY** Sequence classes

Methods that **ARE** attached to Sequence classes

Advanced/Miscellaneous Methods

Functions that are not attached to any Sequence classes

These Functions are not attached to the particular class: they are built-in functions that works with most sequences (and even some sets and mappings type collections).

Functions that may returns a single item or value.

They do not modify the sequence.

```
#We will use the following sequences in this notebook
string_data = 'Hello world!'
list_data = ['0', '3', '2', '1', '2']
# set_data = {'one', 'two', 'three'}  # not a sequence, but still works with
len()
tuple_data = ('0', ['1', '2', '3', '4'], '4')
range_data = range(5)
```

Join()

`join()` is actually a string method that works really well with sequences, as long as **ALL** the items in it are strings. It returns all the items of the seqence joined by a specified pattern.

```
print(', '.join(string_data))                # join the characters in the
string with ', ' as separator

print(' '.join(list_data))                   # join the items in the list with
', ' as separator


#the following does not work, because the sequence has non-string items

#print(', '.join(tuple))

#print(', '.join([0, 3, 2, 1, 4])

#print(', '.join(range_data))
```

```
H, e, l, l, o,  , w, o, r, l, d, !
0 3 2 1 2
```

Repetition

This operation returns a new sequence by repeating the items in a sequence.

For meaningful result the repetition factor must be a positive integer

```
print(2 * string_data)           # repeat the string 2 times
print(string_data * 2)           # repeat the string 2 times
print(0 * tuple_data)           # repeat the list 0 times
print(-2 * list_data)           # repeat the tuple -2 times
```

```
Hello world!Hello world!
Hello world!Hello world!
()
[]
```

Length

The `len()` function will return the number of (top-level) items in a sequence.

```
print(f'string: {len(string_data)}')          # number of character in the
string - 12

print(f'  list: {len(list_data)}')            # number of items in the list -
5
```

```
# print(f'   set: {len(set_data)}')              # number of items in the set -
3

print(f' tuple: {len(tuple_data)}')            # number of items in the tuple -
3

print(f' range: {len(range_data)}')            # number of items in the range -
5
string: 12
  list: 5
 tuple: 3
 range: 5
```

Membership

The `in` keyword returns true if the an item exist in a sequence.

```
print('H' in string_data)                     # check if 'H' is in the string
print('H' not in string_data)                 # check if 'H' is not in the
string
print('h' in string_data)                     # check if 'h' is in the string
print(' ' in string_data)                     # check if ' ' is in the string
print(['1', '2', '3'] in tuple_data)          # check if [1, 2, 3] is in the
tuple
print(2 in range_data)                        # check if 2 is in the range
True
False
False
True
False
True
```

Methods that are attached to Sequence classes

These methods are attached to the particular sequence class.

They do not modify the sequence.

Count

The `count(pattern)` method occurs in all of the sequence classes. It returns the number of occurence of the pattern in the sequence.

```
to_find = 'o'
print(f'number of {to_find}\'s in {string_data} ->
{string_data.count(to_find)}')

to_find = 3
print(f'number of {to_find}\'s in {list(range_data)} ->
{range_data.count(to_find)}')
```

```
to_find = '2'
print(f'number of {to_find}\'s in {list_data} -> {list_data.count(to_find)}')

to_find = '4'
print(f'number of {to_find}\'s in {tuple_data} ->
{tuple_data.count(to_find)}')

tmp = [0, [1, 2], 3, [1, 2], 4, [2, 1]]
to_find = [1, 2]
print(f'number of {to_find}\'s in {tmp} -> {tmp.count(to_find)}')

# there is no count() method for the set class
# to_find = 'two'
# print(f'number of {to_find}\'s in {set_data} -> {set_data.count(to_find)}')
number of o's in Hello world! -> 2
number of 3's in [0, 1, 2, 3, 4] -> 1
number of 2's in ['0', '3', '2', '1', '2'] -> 2
number of 4's in ('0', ['1', '2', '3', '4'], '4') -> 1
number of [1, 2]'s in [0, [1, 2], 3, [1, 2], 4, [2, 1]] -> 2
```

Index

The `index(pattern)` function returns the position of the first occurence of pattern in the sequence.

```
to_find = 'o'
print(f'position of {to_find} in {string_data} ->
{string_data.index(to_find)}')

to_find = 3
print(f'position of {to_find} in {list(range_data)} ->
{range_data.index(to_find)}')

to_find = '2'
print(f'position of {to_find} in {list_data} -> {list_data.index(to_find)}')

to_find = '4'
print(f'position of {to_find} in {tuple_data} ->
{tuple_data.index(to_find)}')

tmp = [0, [1, 2], 3, [1, 2], 4, [2, 1]]
to_find = [1, 2]
print(f'position of {to_find} in {tmp} -> {tmp.index(to_find)}')

# there is no index() method for the set class
# to_find = 'two'
# print(f'position of {to_find} in {set_data} -> {set_data.index(to_find)}')
position of o in Hello world! -> 4
position of 3 in [0, 1, 2, 3, 4] -> 3
position of 2 in ['0', '3', '2', '1', '2'] -> 2
position of 4 in ('0', ['1', '2', '3', '4'], '4') -> 2
position of [1, 2] in [0, [1, 2], 3, [1, 2], 4, [2, 1]] -> 1
```

Mutating Methods

Because only list are mutable, these type of methods were covered in the notebook on list.

Aggregation functions

The main agregation functions are `min()`, `max()` and `sum()`. These functions will process all the items (MUST be numeric) in a sequence and return a single value.

As expected, the sum() and sorted() function will only work if **ALL** the items in a sequence are numbers

```
x = [4, 2, 6, 1]

y = (8, 4, 5)

z = {8, 1.14, 3.14159}

print('Working with string')
print(f'max: \'{max(string_data)}\'')                # maximum character in
the string
print(f'min: \'{min(string_data)}\'')                # minimum character in
the string
print(sorted(string_data))                           # sorts the characters in
the string

print('\nWorking with integers in a list')
print(f'max: {max(x)}')                              # max value in the list
print(f'min: {min(x)}')                              # min value in the list
print(f'sum: {sum(x)}')                              # sum of the values in a
list
print(sorted(x))                                     # sorted list

print('\nWorking with numerics in a tuple')
print(f'max: {max(z)}')                              # max value in a tuple
print(f'min: {min(z)}')                              # min value in a tuple
print(f'sum: {sum(z)}')                              # sum of the values in a
tuple
print(sorted(z))                                     # sorted tuple

print('\nWorking with a range object')
print(f'max: {max(range_data)}')                     # max value in a range
object
print(f'min: {min(range_data)}')                     # min value in a range
object
print(f'sum: {sum(range_data)}')                     # sum of the values in a
range object
print(sorted(range_data))                            # sorted range object

#print(sum(c)) # This will raise an error because one of the items in c is
not number (it is a list!)
Working with string
max: 'w'
min: ' '
```

```
[' ', '!', 'H', 'd', 'e', 'l', 'l', 'l', 'o', 'o', 'r', 'w']

Working with integers in a list
max: 6
min: 1
sum: 13
[1, 2, 4, 6]

Working with numerics in a tuple
max: 8
min: 1.14
sum: 12.281590000000001
[1.14, 3.14159, 8]

Working with a range object
max: 4
min: 0
sum: 10
[0, 1, 2, 3, 4]
```

Concatenation

The concatenation operation (+) returns a new sequence by accumulating the items in two sequencs of the same type.

```
e = ' Centennial College.'
f =[-1, '8', 3, (2, 1), 4]
g = (0, )
h = {'z', 'y', 'x'}

print(string_data + e)
print(list_data + f)
#print(list_data + g)            #  This will raise an error because
concatenation between list and set is not supported
Hello world! Centennial College.
['0', '3', '2', '1', '2', -1, '8', 3, (2, 1), 4]
```

Iteration

This operation returns each item in a sequence for processing.

```
# iteration a string
for x in string_data:
    print(x, end=' ')  # print each character in the string with a space
print()  # print a new line


#iterating a list
b = [0, 3, 2, 1, 4]
total = 0
for x in b:
    total += x  # total all items in the list
print(f'Total: {total}')  # print the total of the list
```

```
print()  # print a new line


#iterating a tuple
for x in tuple_data:
    print(x, end=' ')  # print each item in the tuple with a space
print()  # print a new line

#iertating a range object
for x in range_data:
    print(x, end=' ')  # print each item in the set with a space
print()  # print a new line
H e l l o   w o r l d !
Total: 10

0 ['1', '2', '3', '4'] 4
0 1 2 3 4
```

Indexing

This operation returns an item of a sequence.

In addition to normal indexing, negative indices are permitted. A negative index references elements from the end of the sequence.

```
    String:  H  e  l  l  o

    Index:   0  1  2  3  4

    Negidx: -5 -4 -3 -2 -1
```

```
print(string_data)
print(f'index  0 -> {string_data[0]}')  # access the first character in the
string
print(f'index  1 -> {string_data[1]}')  # access the second character in the
string
print(f'index  2 -> {string_data[2]}')  # access the third character in the
string
print(f'index  3 -> {string_data[3]}')
print(f'index  4 -> {string_data[4]}')
print(f'index  5 -> {string_data[5]}')
print(f'index  6 -> {string_data[6]}')
print(f'index -6 -> {string_data[-6]}')
print(f'index -5 -> {string_data[-5]}')
print(f'index -4 -> {string_data[-4]}')
print(f'index -3 -> {string_data[-3]}')  # access the last but two characters
in the string
print(f'index -2 -> {string_data[-2]}')  # access the last but one character
in the string
print(f'index -1 -> {string_data[-1]}')  # access the last character in the
string
Hello world!
```

```
index  0 -> H
index  1 -> e
index  2 -> l
index  3 -> l
index  4 -> o
index  5 ->
index  6 -> w
index -6 -> w
index -5 -> o
index -4 -> r
index -3 -> l
index -2 -> d
index -1 -> !
```

Slicing

This operation returns a new sequence from a portion of a sequence. This is a difficult concepts to master (atleast I found this so!)

```
a = 'HelloWorld'
for x, i in enumerate(a):
    print(f'{x}:{i}', end=', ')                                  # print each
character in the string with its index
print()  # print a new line

start = 2   # start index
end = 7    # end index
step = 2    # step value
print(f'slice :: -> {a[::]}')                                   # slice the
string from the start to the end
print(f'slice : -> {a[:]}')                                     # slice the
string from the start to the end
print(f'slice {start}: -> {a[start:]}')                         # slice the
string from index 2 to the end
print(f'slice {start}:{end} -> {a[start:end]}')                 # slice the
string from index 2 to 10 (not including 10)
print(f'slice {start}:{end}:{step} -> {a[start:end:step]}') # slice the
string from index 2 to 10 in step 2 (not including 10)
print(f'slice :{end}: -> {a[:end:]}')                           # slice the
string from the start to index 10 (not including 10)
print(f'slice ::{step} -> {a[::step]}')                         # slice the
string from the start to the end in step 2
print(f'slice {start}::{step} -> {a[start::step]}')                      #
slice the string from index 2 to the end in step 2
start = -1
step = -1
print(f'slice {start}::{step} -> {a[start::step]}')             # slice the
string from end to the beginning i.e. reverse the string
```

```
0:H, 1:e, 2:l, 3:l, 4:o, 5:W, 6:o, 7:r, 8:l, 9:d,
slice :: -> HelloWorld
slice : -> HelloWorld
slice 2: -> lloWorld
slice 2:7 -> lloWo
```

```
slice 2:7:2 -> loo
slice :7: -> HelloWo
slice ::2 -> Hlool
slice 2::2 -> lool
slice -1::-1 -> dlroWolleH
```

The zip() function

The zip() function combines elements from multiple iterables (like lists, tuples, dictionaries, etc.) into a single iterable of tuples.

Syntax: python `zip(*iterables)` *iterables: Two or more iterables (e.g., lists, tuples, strings).

In Python version 2, zip returns a list
In python version 3, zip returns an iterable

How It Works:

- zip() pairs the first element of each iterable together, then the second, and so on.
- The result is an iterator of tuples where the i-th tuple contains the i-th element from each input iterable.
- If the input iterables are of unequal length, zip() stops when the shortest iterable is exhausted.
- It's memory efficient since it generates items on-demand
- You can zip any number of iterables together

```python
# The zip function combines multiple iterables into a single iterable of
tuples.
# Each tuple contains one element from each of the input iterables.
# If the input iterables are of different lengths, the resulting iterable
will be as long as the shortest input iterable.
# After the first iteration, the zip object is exhausted and cannot be
reused.
# If you want to use the zip object again, you need to create it again.
# In this example, we will create a zip object with different types of
iterables.

a = zip(
    'hello',                    # a string
    [2, 3, 5, 7, 11, 13],       # a list of prime numbers
    (1, 2, 3, 4),               # a tuple of numbers
    {'one', 'two', 'three', 'four', 'five'},     # a set of strings
    range(4))                   # a range object from 0 to 3
                                # the last argument will be the limiter for
this zip
```

```
                                # and the function will stop after combining
the fourth item
                                # in each of the arguemtns

print(type(a))              # print the type of the zip object
print(list(a))
print(list(a))              #   second print does not work
<class 'zip'>
[('h', 2, 1, 'four', 0), ('e', 3, 2, 'one', 1), ('l', 5, 3, 'three', 2), ('l'
, 7, 4, 'five', 3)]
[]
names = 'Alice Bob Claire Dave Eve'.split()     # split the string into a
list of names
ages = [25, 30, 22, 35, 28]                     # list of ages corresponding
to the names

for x in zip(names):
    print(f'{x}', end=', ')                     # print each name in the list
with a comma and space
('Alice',), ('Bob',), ('Claire',), ('Dave',), ('Eve',),
for name, age in zip(names, ages):
    print(f'{name} is {age} years old')  # print each character in the first
string with its corresponding character in the second string
# create a dictionary from the names and ages
people_dict = dict(zip(names, ages))
print(people_dict)                              # print the resulting
dictionary

age_dict = dict(zip(ages, names))
print(age_dict)                                 # print the resulting
dictionary
{'Alice': 25, 'Bob': 30, 'Claire': 22, 'Dave': 35, 'Eve': 28}
{25: 'Alice', 30: 'Bob', 22: 'Claire', 35: 'Dave', 28: 'Eve'}
# creating a set from a zip iterable
people_set = set(zip(names, ages))
print(people_set)                               # print the resulting set
{('Eve', 28), ('Alice', 25), ('Dave', 35), ('Bob', 30), ('Claire', 22)}
# use the * operator to unpack the zip object (or a list of tuples or a list
of lists )
zipped = [(1, 'a'), (2, 'b'), (3, 'c')]
list1, list2 = zip(*zipped)
print(list1)  # Output: (1, 2, 3)
print(list2)  # Output: ('a', 'b', 'c')

zipped = [[1, 'a', 10], [2, 'b', 20], [3, 'c', 30]]
list1, list2, list3 = zip(*zipped)
print(list1)  # Output: (1, 2, 3)
print(list2)  # Output: ('a', 'b', 'c')
print(list3)  # Output: (10, 20, 30)
(1, 2, 3)
('a', 'b', 'c')
(1, 2, 3)
('a', 'b', 'c')
```

```
(10, 20, 30)
dat = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]   # a list of numbers from 0 to 8
for x in zip(dat, dat[-1::-1]):     # reverse the second sequence
    print(x)                        # Output: (0, 8), (1, 7), (2, 6)


#dat = [1, 2, 3, 4, 5]   # a list of numbers from 0 to 8
list1, list2, list3 = zip(*[iter(dat)]*3)
print(list1)  # Output: (0, 3, 6)
print(list2)  # Output: (1, 4, 7)
print(list3)  # Output: (2, 5, 8)
(0, 10)
(1, 9)
(2, 8)
(3, 7)
(4, 6)
(5, 5)
(6, 4)
(7, 3)
(8, 2)
(9, 1)
(10, 0)
(0, 1, 2)
(3, 4, 5)
(6, 7, 8)
```

Decomposition

```
colors = ('red', 'green', 'blue')
first, middle, last = colors

print(first)                    # red
print (middle)                  # green
print(last)                     # blue
red
green
blue
first, *last = colors
print(first)                    # red
print(last)                     # ['green', 'blue']
red
['green', 'blue']
*first, last = colors
print(first)                    # ['red', 'green']
print(last)                     # blue
['red', 'green']
blue
colors = 'red green blue yellow'.split()
first, *_, last = colors        # the discard is used to silence the linter
print(first)                    # red
print(last)                     # yellow
red
yellow
```

```
grades = ['Alice', 85, 90, 78, 92]

student, *scores = grades
print(student)                    # Alice
print(scores)                     # [85, 90, 78, 92]
Alice
[85, 90, 78, 92]
# Nested unpacking
student = ("Alice", (85, 90, 78))
name, (math, science, english) = student
print(name, math, science, english)
Alice 85 90 78
```

Comparison of the sequences

Sequence	Methods directly associated with the class
tuple	count, index
range	count, index, start, stop, step
list	append, clear, copy, count, extend, index, insert, pop, remove, reverse, sort
str	capitalize, casefold, center, count, encode, endswith, expandtabs, find, format, format_map, index, isalnum, isalpha, isascii, isdecimal, isdigit, isidentifier, islower, isnumeric, isprintable, isspace, istitle, isupper, join, ljust, lower, lstrip, maketrans, partition, removeprefix, removesuffix, replace, rfind, rindex, rjust, rpartition, rsplit, rstrip, split, splitlines, startswith, strip, swapcase, title, translate, upper, zfill

Summary

- Sequences: str, list, tuple, range
- Common operations: len(), indexing, slicing, concatenation, repetition, iteration, membership
- Aggregates: min(), max(), sum(), sorted()
- zip(): combines multiple sequences
- Decomposition: unpack sequence items into variables

Mutating Methods (modify the list)

- `append(x)` → add one item at the end
- `extend(iterable)` → add all items from another iterable
- `insert(i, x)` → insert at a specific index
- `remove(x)` → remove first occurrence of x
- `pop([i])` → remove and return item (default: last)
- `reverse()` → reverse in place
- `sort()` → sort in place
- `clear()` → remove all items

Non-Mutating Methods (return info or copies)

- `copy()` → shallow copy of list
- Aggregate functions such as:
  - `count(x)` → number of occurrences
  - `max(x)` → number of occurrences
  - `min(x)` → number of occurrences
- `index(x)` → position of first occurrence
- (also `len(lst)` and `sorted(lst)` are useful built-ins)